

EVALUACIÓN NUMÉRICA DE LA CONJETURA DE OERTEL

Consuelo Rodríguez Quezada
Profesor: David Salas

1. CONTEXTUALIZACIÓN

En el campo de la geometría convexa, el análisis de cómo un hiperplano divide el volumen de un cuerpo convexo cuenta con resultados clásicos. Uno de los más destacados es el teorema de Grünbaum (1960), el cual dice que por cada cuerpo convexo $C \subset \mathbb{R}^d$, y para cada semiespacio H de $\mathbb{R}^d$, se tiene que

$$H \text{ contiene el centroide de } C \implies \frac{\text{vol}_d(H \cap C)}{\text{vol}_d(C)} \geq \left(\frac{d}{d+1}\right)^d \geq \frac{1}{e}$$

A este resultado se le conoce como la desigualdad de Grünbaum.

Más recientemente, Oertel ha propuesto una conjetura que aplica similarmente para cuerpos entero-mixtos, planteando que existe un punto de apoyo, conocido como *centerpoint* que garantiza una fracción mínima de volumen tras ser cortado por cualquier hiperplano; de la siguiente manera

$$\mathcal{F}(S) := \max_{x \in S} (\inf\{\mu(H \cap S) : H \text{ es semiespacio con } x \in H\})$$

A esta función le llamamos radio de Oertel.

Es presente informe detalla el desarrollo e implementación de una metodología determinista para la evaluación numérica de Oertel. El principal aporte metodológica radica en una representación por vértices de la figura convexa, y en la operación numérica, no estimativa, de las áreas de esta figura.

1.1. Alcance y restricciones. El alcance de este documento comprende el análisis del caso de una dimensión discreta y dos continuas ($n = 1, d = 2$), además del procesamiento de tres fibras en la dimensión entera ($z = 0, 1, 2$).

El código fue desarrollado con Python 3, utilizando principalmente la librería "*NumPy*" para operaciones algebraicas vectorizadas, y "*SciPy*" para la extracción de las envolventes convexas en la proyección 2D, con la función [ConvexHull](#).

1.2. Estructura del informe. El documento se estructura de la siguiente manera: la sección 2 presenta los algoritmos y una descripción escrita para cada función utilizada. Posteriormente, en la sección 3 se detalla el análisis de resultados obtenidos al lanzar pruebas numéricas bajo 2 casos: 5 puntos por fibra, y 10 puntos por fibra.

2. FUNCIONES

Algorithm 1 Generación de vértices aleatorios por fibra

Input: z_vals (valores de z), d (dimensión), n_per_z (cantidad de puntos)

Output: $verts$ (Arreglo con los vértices concatenados)

```

1:  $blocks \leftarrow$  lista vacía
2: for cada  $z$  en  $z\_vals$  do
3:    $p \leftarrow$  generar  $n\_per\_z$  puntos aleatorios en  $[0, 1]^d$ 
4:    $zcol \leftarrow$  crear columna de tamaño  $n\_per\_z$  con valor  $z$ 
5:    $bloque\_actual \leftarrow$  concatenar horizontalmente ( $zcol \parallel p$ )
6:   Agregar  $bloque\_actual$  a  $blocks$ 
7: end for
8:  $verts \leftarrow$  concatenar verticalmente todos los elementos de  $blocks$ 
9: return  $verts$ 

```

random_vertices_by_fiber Por cada z en z_vals genera la cantidad de puntos aleatorios que necesitamos, en un rango de $[0, 1]^d$, y luego lo concatena horizontalmente con una columna de valores iguales al z en el que estamos. Finalmente, concatena verticalmente todos los elementos para formar el array.

Algorithm 2 Conversión de H-representación a V-representación

Input: A (Matriz de coeficientes de tamaño $m \times n$), b (Vector de tamaño m), tol (Tolerancia numérica)

Output: $vertices$ (Arreglo con los vértices del cuerpo convexo)

```

1:  $m, n \leftarrow$  dimensiones de  $A$ 
2:  $vertices \leftarrow$  lista vacía
3: for cada combinación de índices  $I \subset \{1, 2, \dots, m\}$  de tamaño  $n$  do
4:    $A_{sub} \leftarrow$  submatriz de  $A$  tomando las filas en  $I$ 
5:    $b_{sub} \leftarrow$  subvector de  $b$  tomando las entradas en  $I$ 
6:   if  $A_{sub}$  no es cuadrada, o  $|\det(A_{sub})| > tol$  then           ▷ Chequeamos que los hiperplanos no
     sean paralelos
7:      $punto \leftarrow$  solución del sistema lineal  $A_{sub} \cdot x = b_{sub}$ 
8:     if  $A \cdot punto \leq b + tol$  then                               ▷ Verificamos que el punto esté en el polítopo
9:        $new\_point \leftarrow$  Verdadero
10:      for cada  $v$  en  $vertices$  do
11:        if  $v$  es casi igual a  $punto$  (con cierta  $tol$ ) then
12:           $new\_point \leftarrow$  Falso
13:          break                                                   ▷ Rompemos el ciclo si ya existe
14:        end if
15:      end for
16:      if  $new\_point$  es Verdadero then
17:        Agregar  $punto$  a  $vertices$ 
18:      end if
19:    end if
20:  end if
21: end for
22: return  $vertices$  as array

```

h.to_v_rep Esta función convierte la H representación de un polítopo, en una V representación del mismo, buscando los puntos intersección de los planos que conforman la H rep.

Algorithm 3 Ordenamiento de vértices por ángulo relativo por slice

Input: *vertices_slice* (Conjunto de vértices 2D desordenados), *z* (Coordenada de la fibra)

Output: *puntos_ordenados* (Arreglo de vértices 3D ordenados secuencialmente)

```

1: centro  $\leftarrow$  Promedio de las coordenadas de vertices_slice
2: vectores  $\leftarrow$  Lista vacía
3: for cada v  $\in$  vertices_slice do
4:   Agregar (v − centro) a vectores
5: end for
6: uref  $\leftarrow$  vectores[0] ▷ Tomamos el primer vector como referencia
7: puntos_izq  $\leftarrow$  {(uref, 0)} ▷ Diccionario: guarda (vector  $\rightarrow$  ángulo)
8: puntos_der  $\leftarrow$  {}
9: for cada ui en vectores[1...final] do
10:    $\theta_i \leftarrow \arccos\left(\frac{u_{ref} \cdot u_i}{\|u_{ref}\| \|u_i\|}\right)$  ▷ Ángulo en  $[0, \pi]$ 
11:   det  $\leftarrow u_{ref,x} \cdot u_{i,y} - u_{ref,y} \cdot u_{i,x}$  ▷ Producto cruz 2D
12:   if det  $\geq 0$  then ▷ El punto está a la izquierda (antihorario)
13:     puntos_izq[ui]  $\leftarrow \theta_i$ 
14:   else ▷ El punto está a la derecha (horario)
15:     puntos_der[ui]  $\leftarrow \theta_i$ 
16:   end if
17: end for
18: Ordenar puntos_izq de menor a mayor según su ángulo
19: Ordenar puntos_der de menor a mayor según su ángulo
20: puntos_ordenados  $\leftarrow$  lista vacía
21: for cada u en las llaves de puntos_izq ordenado do
22:   p  $\leftarrow u + centro$  ▷ Recuperamos las coordenadas originales
23:   Agregar (z, px, py) a puntos_ordenados
24: end for
25: for cada u en las llaves de puntos_der en orden inverso do
26:   p  $\leftarrow u + centro$ 
27:   Agregar (z, px, py) a puntos_ordenados
28: end for
29: return puntos_ordenados as array

```

ordenar_vértices Se elige un punto dentro de la figura como centro, en este caso el promedio de los vértices (como es un cuerpo convexo, cualquier punto interior a este puede definirse por el promedio de los vértices ya que es también una combinación convexa); luego se ocupa este punto para trazar vectores desde él hasta cada vértice, y se toma "el primer" vector como referencia para obtener el ángulo respectivo a este y cada otro vector. Finalmente se ordenan los vectores por ángulos de menor a mayor, y asimismo se ordena los puntos respectivos para obtener un array ordenado, anticlockwise, de puntos en 3D. Se ordenan en 2D las coordenadas continuas, y al final se agrega al principio la coordenada *z* correspondiente.

Algorithm 4 Cálculo del centroide de un polígono 2D (Candidato a Centerpoint)

Input: *vertices* (Arreglo de vértices ordenados secuencialmente), *z* (Coordenada de la fibra)

Output: *centroide* (Punto 3D candidato a centerpoint)

```

1:  $n \leftarrow$  cantidad de puntos en vertices
2:  $area \leftarrow 0$ 
3:  $v_x \leftarrow 0, v_y \leftarrow 0$ 
4: for  $i = 0$  hasta  $n - 1$  do
5:    $(z_0, x_0, y_0) \leftarrow vertices[i]$ 
6:    $(z_1, x_1, y_1) \leftarrow vertices[(i + 1) \bmod n]$   $\triangleright$  El último vértice se conecta con el primero
7:    $A \leftarrow (x_0 \cdot y_1 - x_1 \cdot y_0)$   $\triangleright$  Término de la fórmula de Gauss / Teorema de Green
8:    $area \leftarrow area + A$ 
9:    $v_x \leftarrow v_x + A \cdot (x_0 + x_1)$ 
10:   $v_y \leftarrow v_y + A \cdot (y_0 + y_1)$ 
11: end for
12:  $area \leftarrow \frac{area}{2}$ 
13:  $v_x \leftarrow \frac{v_x}{6 \cdot area}$ 
14:  $v_y \leftarrow \frac{v_y}{6 \cdot area}$ 
15: return  $(z, v_x, v_y)$  as array

```

get_centroid Esta función obtiene el centroide de un polígono utilizando los vértices del cuerpo, según el Teorema de Green, para lo cual los vértices en el input deben estar ordenados secuencialmente; y retorna el centroide como punto con 3 coordenadas.

Algorithm 5 Generación de línea de ensayo (Muestreo en segmento de recta)

Input: v_1, v_a (Puntos extremos del segmento en $\mathbb{R}^d$), $N_Muestras$ (Cantidad de puntos, por defecto 50)

Output: *muestras* (Arreglo con los puntos generados sobre el segmento)

```

1:  $t\_vals \leftarrow$  Generar secuencia de  $N\_Muestras$  valores equidistantes en el intervalo  $(0, 1)$ 
2:  $muestras \leftarrow$  lista vacía
3: for cada  $t$  en  $t\_vals$  do
4:    $muestra \leftarrow (1 - t) \cdot v_1 + t \cdot v_a$   $\triangleright$  Combinación convexa pura
5:   Agregar muestra a muestras
6: end for
7: return muestras as array

```

linea_de_ensayo Esta función crea una secuencia de puntos equidistantes entre dos puntos estratégicos para el muestreo de posibles centerpoints.

Algorithm 6 Obtención de puntos candidatos a Centerpoint

Input: *vertices* (Conjunto de todos los vértices del polítopo en $\mathbb{R}^3$), *n_muestras* (Resolución de la línea de ensayo)

Output: *candidatos* (Lista de puntos a evaluar), *ordenados_todos* (Lista de polígonos 2D por fibra)

```

1: candidatos  $\leftarrow$  lista vacía
2: ordenados_todos  $\leftarrow$  lista vacía
3:  $P_0 \leftarrow \{v \in \text{vertices} \mid v_z \approx 0\}$ 
4:  $P_2 \leftarrow \{v \in \text{vertices} \mid v_z \approx 2\}$ 
5:  $Q \leftarrow \{\frac{p_0+p_2}{2} \mid p_0 \in P_0, p_2 \in P_2\}$  ▷ Puntos medios entre las fibras extremas
6: for cada  $z \in \{0, 1, 2\}$  do
7:   grupo  $\leftarrow \{v \in \text{vertices} \mid v_z \approx z\}$ 
8:   if  $z = 1$  then
9:     grupo  $\leftarrow \text{grupo} \cup Q$  ▷ Inyectamos  $Q$  en la fibra discreta central
10:  end if
11:  grupo_2d  $\leftarrow$  Proyectar grupo a 2D (omitir coordenada  $z$ )
12:  hull  $\leftarrow$  EnvolventeConvexa2D(grupo_2d) ▷ Nos quedamos solo con la frontera
13:  ordenados  $\leftarrow$  ordenar_vertices(hull,  $z$ )
14:  Agregar ordenados a ordenados_todos
15:  candidato  $\leftarrow$  get_centroid(ordenados,  $z$ )
16:  Agregar candidato a candidatos
17: end for
18:  $v_a \leftarrow \frac{\text{candidatos}[0] + \text{candidatos}[2]}{2}$  ▷ Punto medio de los centroides 0 y 2
19: if  $v_a$  es casi igual a candidatos[1] then
20:   return candidatos, ordenados_todos
21: else
22:   Agregar  $v_a$  a candidatos ▷ El índice de  $v_a$  pasa a ser 3
23:   muestras  $\leftarrow$  linea_de_ensayo(candidatos[1],  $v_a$ , n_muestras)
24:   Añadir todos los elementos de muestras a la lista candidatos
25:   return candidatos, ordenados_todos
26: end if

```

obtener_candidatos A partir de los vértices que conforman el cuerpo convexo, esta función entrega una lista con los candidatos a centerpoint de la figura. Primero, itera por cada fibra, obtiene la envolvente convexa y ordena los puntos en orden antihorario con *ordenar_vertices*, luego con la función *get_centroid* entrega el centroide de la cara sobre la fibra que está iterando. Siguiendo, luego de iterar para $z = 0, 1$ y 2 , obtuvimos los centroides v_0, v_1 , y v_2 . Luego, obtenemos un cuarto punto candidato, que viene del punto promedio entre v_0 y v_2 , llamado v_a . Y finalmente, aplicamos la función *linea_de_ensayo* para obtener más candidatos entre los puntos v_1 y v_a .

Algorithm 7 Cálculo de área exacta de un polígono 2D

Input: *grupo* (Arreglo de vértices 2D ordenados secuencialmente)

Output: *area* (Área exacta del polígono)

```

1:  $n \leftarrow$  cantidad de vértices en grupo
2:  $area \leftarrow 0$ 
3: for  $i = 0$  hasta  $n - 1$  do
4:    $(x_0, y_0) \leftarrow grupo[i]$ 
5:    $(x_1, y_1) \leftarrow grupo[(i + 1) \bmod n]$ 
6:    $A \leftarrow (x_0 \cdot y_1 - x_1 \cdot y_0)$ 
7:    $area \leftarrow area + A$ 
8: end for
9:  $area \leftarrow area \cdot 0.5$ 
10: return area

```

area_2d A partir de un arreglo 2D de vértices, correspondientes a una fibra z , esta función nos entrega el área total 2D.

Algorithm 8 Cálculo del volumen total exacto (Suma de áreas por rebanada)

Input: *vertices* (Conjunto de vértices 3D), *z_vals* (Fibras a evaluar, defecto $\{0, 1, 2\}$)

Output: *sum_area* (Volumen total aproximado por las rebanadas discretas)

```

1:  $sum\_area \leftarrow 0$ 
2: for cada  $z \in z\_vals$  do
3:    $grupo \leftarrow \{v \in vertices \mid v_z \approx z\}$ 
4:    $grupo\_2d \leftarrow$  Proyectar grupo extrayendo las coordenadas  $(x, y)$ 
5:    $n \leftarrow$  cantidad de puntos en grupo_2d
6:    $area \leftarrow 0$ 
7:   for  $i = 0$  hasta  $n - 1$  do
8:      $(x_0, y_0) \leftarrow grupo\_2d[i]$ 
9:      $(x_1, y_1) \leftarrow grupo\_2d[(i + 1) \bmod n]$ 
10:     $A \leftarrow (x_0 \cdot y_1 - x_1 \cdot y_0)$ 
11:     $area \leftarrow area + A$ 
12:   end for
13:    $area \leftarrow |area \cdot 0.5|$  ▷ Valor absoluto por si el ordenamiento fue horario
14:    $sum\_area \leftarrow sum\_area + area$ 
15: end for
16: return sum_area

```

area_total A partir del conjunto total de vértices de la figura, itera por fibra z , obtiene el área por fibra, y devuelve la suma total de las áreas, conformando el volumen total de la figura.

Algorithm 9 Evaluación de posición respecto al hiperplano de corte

Input: *punto* (Vértice 2D), *z* (Elevación de la rebanada), *cp* (Punto Centerpoint en 3D), *u* (Vector normal del hiperplano)

Output: Distancia con signo (Positivo $\implies$ izquierda/adentro, Negativo $\implies$ derecha/afuera)

1: $x' \leftarrow (z, punto_x, punto_y)$ $\triangleright$ Reconstrucción del punto a 3D

2: **return** $u \cdot (x' - cp)$ $\triangleright$ Producto punto (distancia proyectada)

lado_hiperplano Esta función determina de qué lado del hiperplano está un punto. Se toma el punto, se reconstruye su posición original en el plano 3D adjuntándole su fibra *z* correspondiente, y se le aplica producto punto respecto al centerpoint en 3D de la figura completa.

Algorithm 10 Cálculo de intersección entre arista y el hiperplano

Input: p_0, p_1 (Puntos 2D de la arista), val_0, val_1 (Evaluaciones de p_0 y p_1 según **Algorithm 9**)

Output: p_{inter} (Punto 2D de intersección)

1: $t \leftarrow \frac{val_0}{val_0 - val_1}$ $\triangleright$ Parámetro de interpolación lineal

2: $p_{inter} \leftarrow p_0 + t \cdot (p_1 - p_0)$ $\triangleright$ Combinación convexa / Interpolación

3: **return** p_{inter}

interseccion Con esta función determinamos dónde se encuentra la intersección de una arista al hiperplano. Para esto, sabemos que la intersección se encuentra a una altura que su producto interno respecto al centerpoint de la figura será cero. Si llamamos temporalmente "valor" al producto interno entre un punto y el centerpoint, entonces, buscamos el peso t que hace que la combinación convexa entre el valor del punto anterior a la intersección y el valor de punto siguiente de la intersección sea cero. Es decir:

$$val_0 + t(val_1 - val_0) = 0$$

De esta manera, despejamos t y reemplazamos en la combinación convexa entre los puntos anterior y siguiente de la intersección ($p_0 + t(p_1 - p_0)$), para saber en qué posición cae esta intersección.

Algorithm 11 Recorte de Polígono por un Hiperplano (Sutherland-Hodgman)

Input: *poligono* (Arreglo de vértices 2D), *z* (Elevación), *cp* (Centerpoint), *u* (Vector normal)

Output: *nuevo_poligono* (Arreglo de vértices 2D del polígono recortado)

```

1:  $n \leftarrow$  cantidad de vértices en poligono
2: nuevo_poligono  $\leftarrow$  lista vacía
3: for  $i = 0$  hasta  $n - 1$  do
4:    $p_{actual} \leftarrow poligono[i]$ 
5:    $p_{next} \leftarrow poligono[(i + 1) \bmod n]$ 
6:    $val_{actual} \leftarrow lado\_hiperplano(p_{actual}, z, cp, u)$ 
7:    $val_{next} \leftarrow lado\_hiperplano(p_{next}, z, cp, u)$ 
8:   if  $val_{actual} \geq 0$  y  $val_{next} \geq 0$  then                                ▷ Ambos puntos están adentro
9:     Agregar  $p_{next}$  a nuevo_poligono
10:  else if  $val_{actual} \geq 0$  y  $val_{next} < 0$  then                                ▷ Sale del semiespacio
11:     $inter \leftarrow interseccion(p_{actual}, p_{next}, val_{actual}, val_{next})$ 
12:    Agregar inter a nuevo_poligono
13:  else if  $val_{actual} < 0$  y  $val_{next} \geq 0$  then                                ▷ Entra al semiespacio
14:     $inter \leftarrow interseccion(p_{actual}, p_{next}, val_{actual}, val_{next})$ 
15:    Agregar inter a nuevo_poligono
16:    Agregar  $p_{next}$  a nuevo_poligono
17:  end if                                ▷ Si ambos son  $< 0$ , están afuera y se ignoran
18: end for
19: return nuevo_poligono
  
```

clip_poligono Con esta función cortamos una cara de la figura, recuperando su lado positivo, o "dentro" del hiperplano. A este algoritmo se le conoce como algoritmo de Sutherland-Hodgman, y establece el procedimiento para ir vértice a vértice (en orden) determinando si pertenecen al conjunto dentro o fuera del hiperplano.

Primero, iteramos por cada punto, definimos el punto actual y el siguiente, además de los "valores" (producto punto respecto al centerpoint) para el punto actual y siguiente. Por cada iteración comparamos respecto al hiperplano, si ambos están sobre este, o si están a lados opuestos de este, y se guardan el punto siguiente o la intersección al nuevo conjunto, dependiendo del caso.

Algorithm 12 Evaluación de la Funcional de Oertel para un candidato (Ratio)

Input: *ordenados* (Polígonos 2D por rebanada en orden antihorario), *cp* (Candidato a centerpoint), N_z, N_d (Resolución angular), *z_vals* (Fibras)

Output: *worst_ratio* (Valor de $\mathcal{F}(cp)$), *worst_u* (Dirección del peor corte)

```

1: tot_area  $\leftarrow$  |area_total(ordenados)|
2: if tot_area  $\leq$  0 then
3:   return 0.0, Nulo
4: end if
5: worst_ratio  $\leftarrow$  1.0
6: worst_u  $\leftarrow$  Nulo
7: for  $i = 0$  hasta  $N_z - 1$  do
8:    $\alpha \leftarrow i \cdot \frac{\pi}{N_z}$ 
9:   for  $j = 0$  hasta  $N_d - 1$  do
10:     $\beta \leftarrow j \cdot \frac{\pi}{N_d}$ 
11:     $u \leftarrow (\cos \beta, \cos \alpha \sin \beta, \sin \alpha \sin \beta)$  ▷ Vector normal en coordenadas esféricas
12:    sum_pos  $\leftarrow$  0, sum_neg  $\leftarrow$  0
13:    for cada  $z \in z\_vals$  do
14:      grupo  $\leftarrow$  extraer polígono 2D de ordenados en la fibra  $z$ 
15:      poly_pos  $\leftarrow$  clip_poligono(grupo,  $z$ , cp,  $u$ )
16:      poly_neg  $\leftarrow$  clip_poligono(grupo,  $z$ , cp,  $-u$ )
17:      sum_pos  $\leftarrow$  sum_pos + |area_2d(poly_pos)|
18:      sum_neg  $\leftarrow$  sum_neg + |area_2d(poly_neg)|
19:    end for
20:    area_minima  $\leftarrow$  min(sum_pos, sum_neg)
21:    ratio  $\leftarrow$   $\frac{area\_minima}{tot\_area}$ 
22:    if ratio < worst_ratio then
23:      worst_ratio  $\leftarrow$  ratio
24:      if sum_pos  $\leq$  sum_neg then
25:        worst_u  $\leftarrow$   $u$ 
26:      else
27:        worst_u  $\leftarrow$   $-u$ 
28:      end if
29:    end if
30:  end for
31: end for
32: return worst_ratio, worst_u

```

ratio Esta función obtiene:

$$\mathcal{F}(cp) = \min_u \frac{\sum_z \min(\text{Vol}(S_z \cap H_u^+), \text{Vol}(S_z \cap H_u^-))}{\sum_z \text{Vol}(S_z)}$$

Con esta función respondemos a cuál es la fracción mínima de volumen que se puede garantizar recuperar si uno se apoya en un centerpoint fijo, sin importar el hiperplano que lo corte; un paso previo muy importante para obtener el radio de Oertel.

Lo que hace esta función es, primero que nada, toma los vértices ordenados y obtiene el área total de la figura. Luego, en un for anidado itera los posibles valores para el hiperplano, y para cada uno utiliza **clip_poligono** para definir el lado positivo y el negativo de la figura; recupera el lado con menor área y le saca el ratio dividiendo por le área total. Si el ratio resulta ser peor al anteriormente definido, lo actualiza junto con el hiperplano.

Algorithm 13 Búsqueda del Centerpoint Óptimo (Evaluación Numérica Final)

Input: *ordenados* (Polígonos 2D por rebanada), *puntos_test* (Candidatos generados por Algoritmo 6)

Output: *best_cp* (Mejor Centerpoint), *best_f* (Máximo valor de $\mathcal{F}$ encontrado), *best_u* (Dirección crítica asociada)

```

1: best_f  $\leftarrow -\infty$ 
2: best_cp  $\leftarrow$  Nulo, best_u  $\leftarrow$  Nulo
3: indice_optimo  $\leftarrow 0$ 
4: for idx = 0 hasta longitud de puntos_test - 1 do
5:   cp  $\leftarrow$  puntos_test[idx]
6:   F_cp, u_cp  $\leftarrow$  ratio(ordenados, cp)           ▷ Evaluamos la funcional usando Algoritmo 12
7:   if F_cp > best_f then
8:     best_f  $\leftarrow$  F_cp
9:     best_cp  $\leftarrow$  cp
10:    best_u  $\leftarrow$  u_cp
11:    indice_optimo  $\leftarrow$  idx
12:   end if
13: end for
14: return best_cp, best_f, best_u, indice_optimo

```

new_oertel Esta función busca finalmente el centerpoint aproximado, maximizando la función anterior. Itera por cada candidato en la lista de candidatos a centerpoint, y evalúa su radio de Oertel con **ratio**, para el cual vamos guardando el que maximice la función, junto con el punto, la dirección y su índice en la lista de candidatos.

3. RESULTADOS Y ANÁLISIS

Se lanzaron pruebas evaluando numéricamente la conjetura de Oertel. Primero, se consideró el caso en que se colocaban 5 puntos aleatorios por cada fibra, y luego el caso en que se lanzaban 10 puntos aleatorios por fibra. A continuación se presentan los resultados.

3.1. **n_per_z = 5.** El siguiente gráfico muestra la distribución de los valores para el radio de Oertel, en el caso con 5 puntos aleatorios por fibra. Como se puede observar, no se llega a la cota establecida; además la distribución se ve sesgada hacia la derecha, con la mayoría de casos llegando a un radio entre 0.4 y 0.5

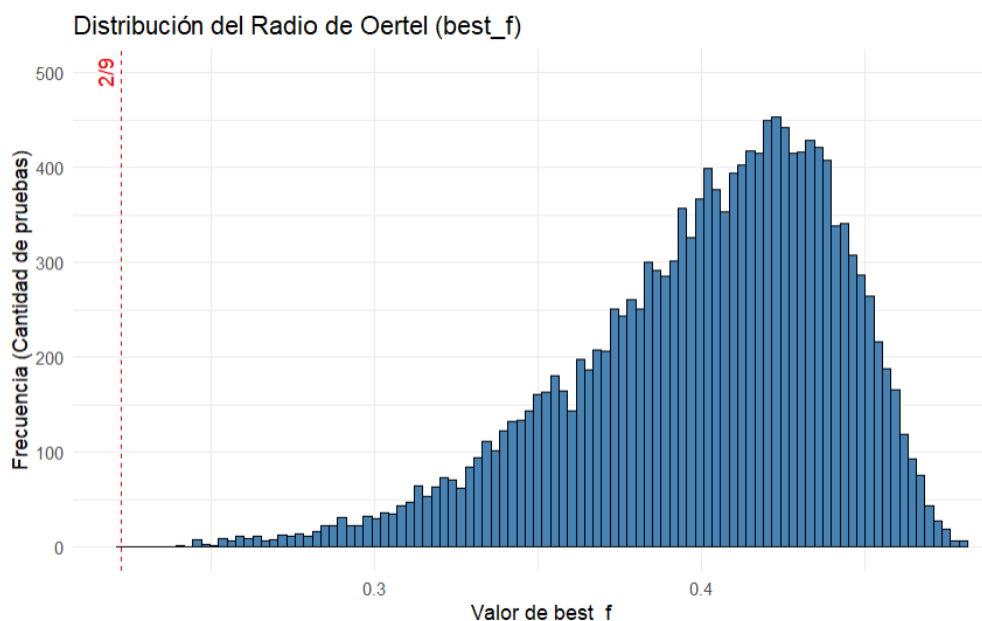


FIGURE 1. Distribución del Ratio de Oertel con $n_per_z = 5$

Luego tenemos un gráfico que nos indica la posición del centerpoint en el 5% de casos con menor radio. En su mayoría se encuentran en la fibra 1.

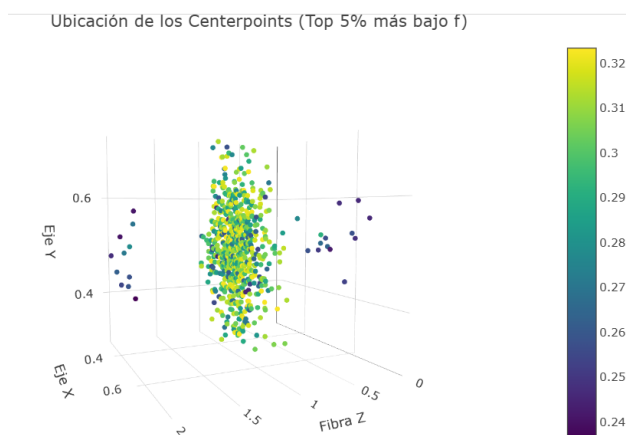


FIGURE 2. Ubicación de los centerpoints con menor ratio, caso $n_per_z = 5$

3.2. $n_{per.z} = 10$. El siguiente gráfico muestra la distribución de los valores para el radio de Oertel, ahora en el caso con 10 puntos aleatorios por fibra. En este caso tampoco se logra llegar a la cota establecida, y además, se observa una mayor densidad de datos entre 0.4 y 0.5 con respecto al caso anterior.

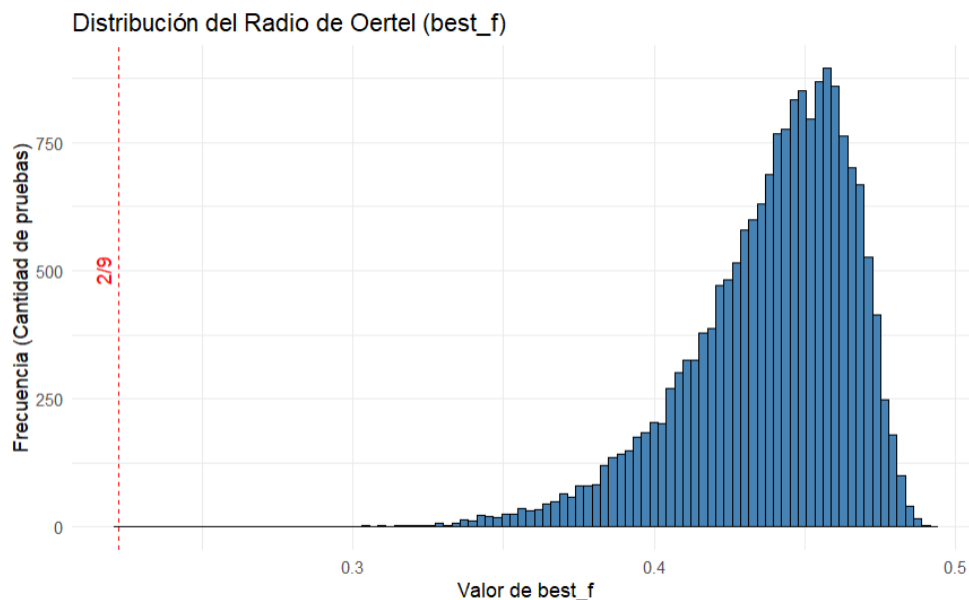


FIGURE 3. Distribución del Ratio de Oertel con $n_{per.z} = 10$

Luego, tenemos un gráfico que nos indica la posición del centerpoint en el 5% de los casos con menor radio. En su totalidad se encuentran en la fibra 1.

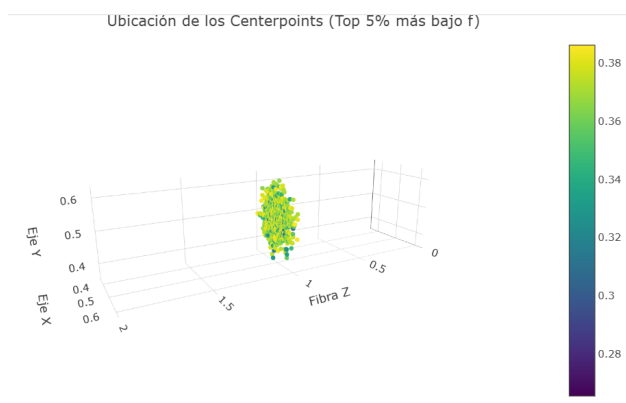


FIGURE 4. Unicación de los centerpoint con menor ratio, caso $n_{per.z} = 10$

Sin embargo, no sólo el 5%, sino que, como se ve en el siguiente gráfico, la totalidad de centerpoints, en este caso, caen en la fibra 1.

Si recordamos, una parte de lo que la función `new_oertel` retorna es la ubicación como índice del centerpoint elegido en la lista de candidatos. Los puntos v_0, v_1, v_2 corresponden a las fibras 0, 1, 2 respectivamente, mientras que el resto de los puntos corresponden a la fibra 1, siendo el punto medio entre las fibras 0 y 2.

Por lo tanto, si nos dirigimos al gráfico, vemos que en su mayoría el punto elegido como centerpoint de la figura es el centroide de la cara en la fibra 1; los centroides de las fibras 0 y 2 no fueron nunca elegidos al tener frecuencia 0, y el resto de los puntos fueron parte de la línea de ensayo de la función [linea_de_ensayo](#).

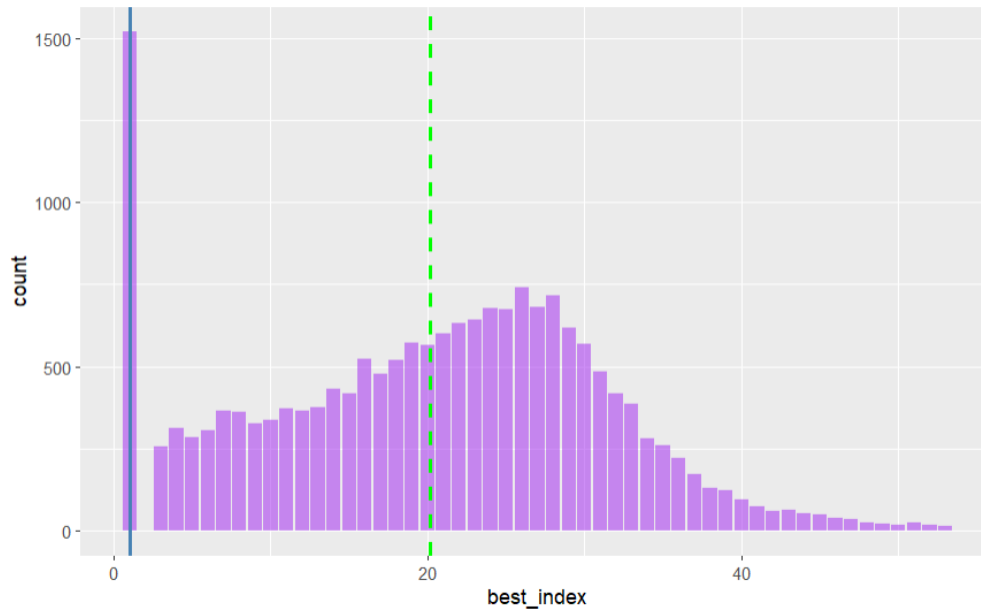


FIGURE 5. Frecuencia de la fibra en que cae cada centerpoint

3.3. Conclusiones. A partir de los resultados obtenidos en las simulaciones, se puede extraer que bajo ningún caso estudiado se ha llegado a contradecir la cota establecida, por lo que existe evidencia que apunta a su veracidad.